

Week 8 – Coordinate frame management using the tf2 library

ROBOTIC EMBEDDED PROGRAMMING
MENG 4060

ANDY ALUBAIDY, PHD, P.ENG.

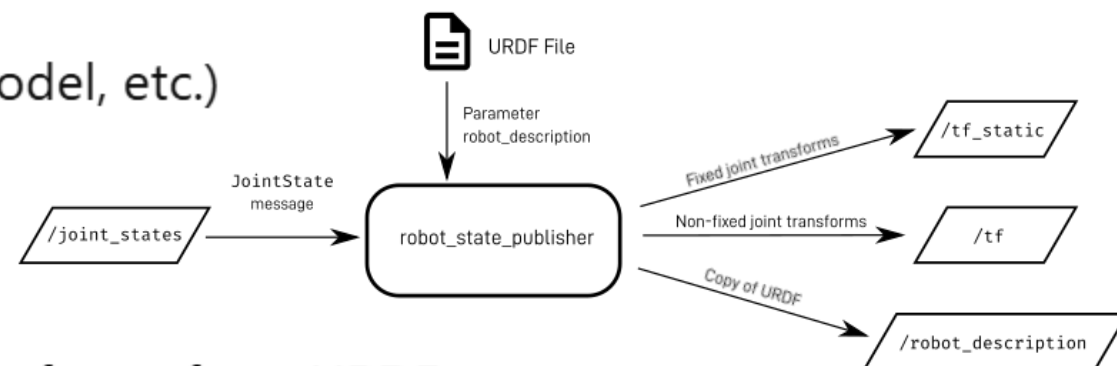
Review from Last Week



- Viewing in RViz
 - Add TF visualisation
 - Set **fixed frame** (e.g., world)
 - Arrows show **child** → **parent** relationships
 - Can configure display (frame names, robot model, etc.)

- **Broadcasting a Moving Robot**

- Use **URDF** to define robot links & joints
- `robot_state_publisher` auto-broadcasts transforms from URDF
- `joint_state_publisher_gui` provides sliders for moving joints
- Visualise in RViz with **TF** and **RobotModel**



Agenda for today's training session



Performance Objective:

- **Implement coordinate frame management using the tf2 library to track and transform reference frames in robotic applications.**
- **Utilize simulation environments such as Gazebo with ROS2.**

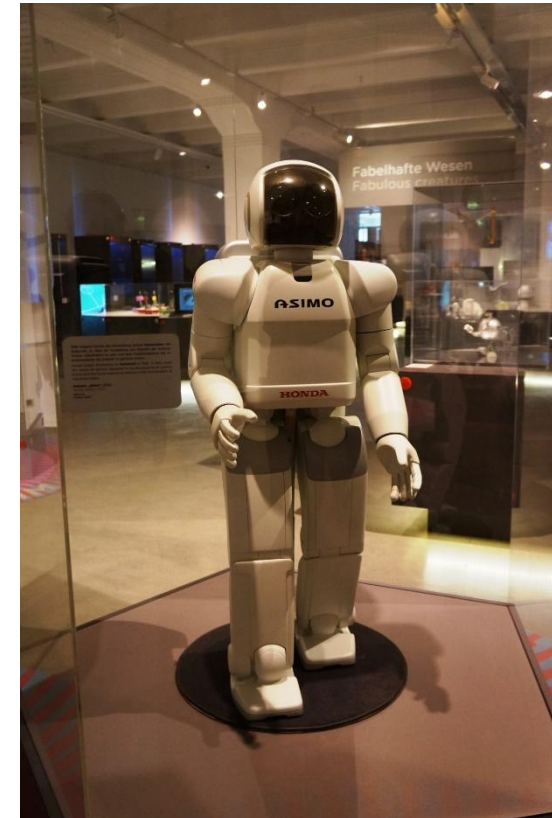
Problem we will be solving today:

- **Unified robot description format –URDF**
 - **Links and Joints**
 - **XML Macros**
 - **Gazebo**

What Is a Robot Description?

- **Multiple software components (e.g., simulation, visualization, control) need robot parameters**
- **To ensure consistency, ROS stores all data in one file**
- **That file: URDF → Unified Robot Description Format**

🧩 URDF = the blueprint of the robot's physical structure



Why URDF?

- Centralized repository of robot geometry and kinematics
- Avoids mismatched definitions across different nodes
- Reusable by:
 - RViz (visualization)
 - Gazebo / Isaac Sim (simulation)
 - MoveIt 2 (motion planning)

💡 Think of URDF as the “digital twin skeleton” of your robot.

First Impressions

- A complete URDF may look **complex**
 - But it's built from a few repeating blocks:
 - **Links** → physical bodies
 - **Joints** → connections between links
 - Once you know the pattern, it's straightforward!
- 🧠 Key idea: *URDF = Links + Joints + XML tags + xacro macros.*

Lecture Roadmap

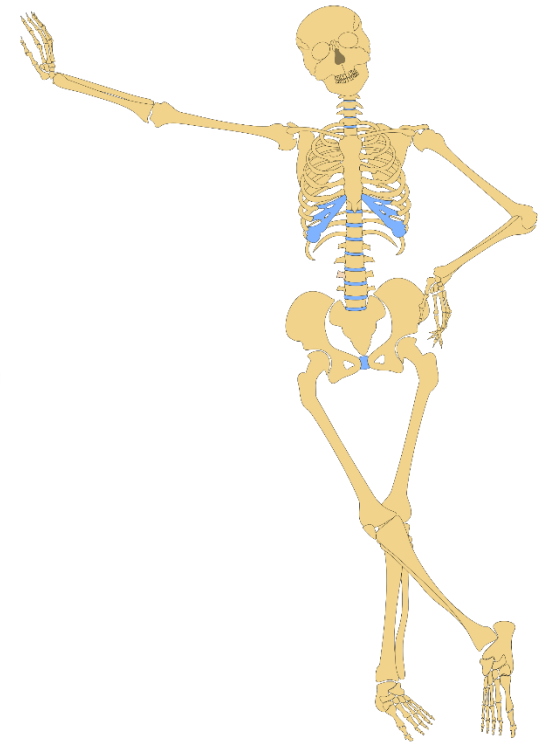
1. Overall structure: Links and Joints
2. The URDF Syntax and robot tag
3. Link and Joint tags
4. Extra tags and naming conventions
5. Introducing xacro (macros and reusability)



Overall Structure: Links and Joints

- URDF describes a **robot as a tree** structure
- **Links** → physical components (bodies)
- **Joints** → define how one link moves relative to another
- Together, they describe **the robot's geometry and kinematics**

✚ Think: *"Links = bones, Joints = movement."*

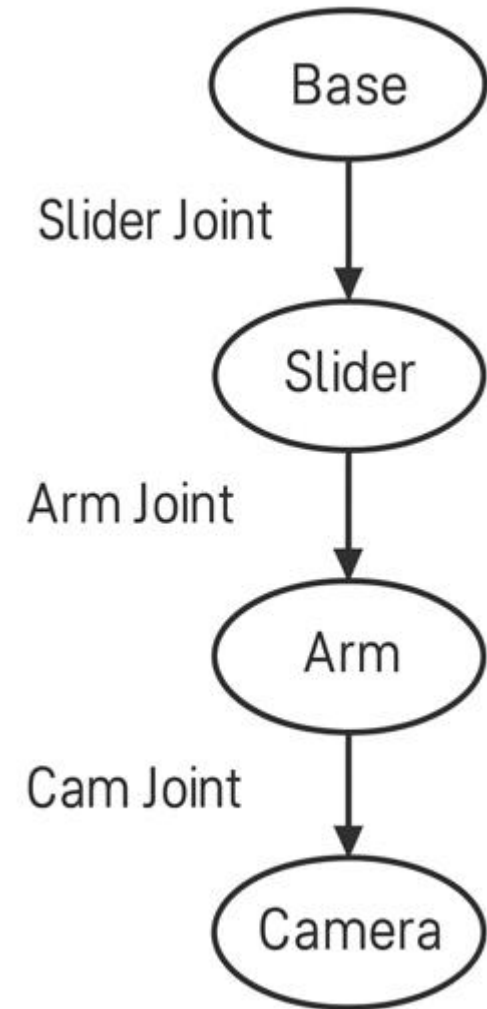
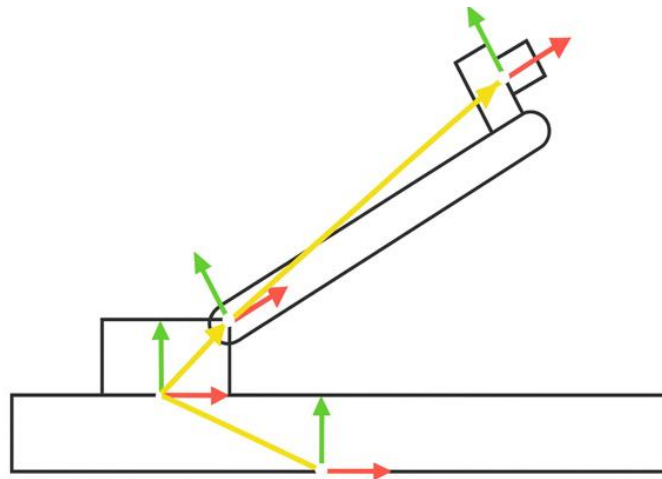


The URDF Tree

- Each robot begins with a **root link**
- Every additional link is connected to a parent link via a **joint**
- The structure forms a **directed tree** (no cycles)

Example:

Base → shoulder → elbow → wrist → gripper

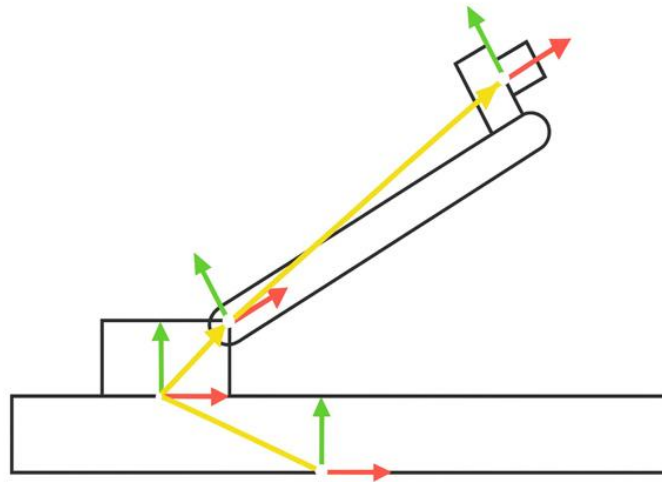


When to Create a Link or Joint

Define a **new link/joint** when:

1. One part **moves relative** to another (e.g., arm segment)
2. A part is **static but needs a reference** (e.g., camera or LiDAR position)

🎯 *Helps keep transforms organized and sensors aligned in simulation.*



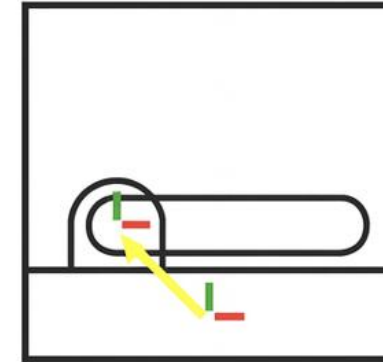
Types of URDF Joints

Joint Type	Motion	Example Use	Limits
Revolute	Rotational	Arm joint	Yes (angular range)
Continuous	Unlimited rotation	Wheel	No limit
Prismatic	Linear slide	Telescopic arm	Yes (linear range)
Fixed	No motion	Sensor mount	None

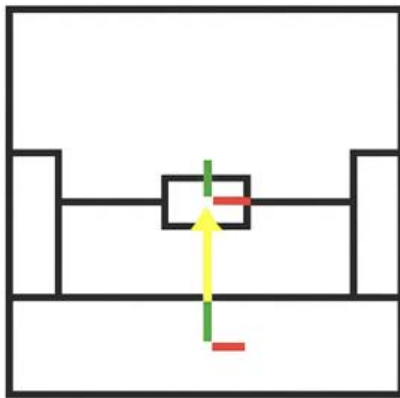
 Choose the simplest type that matches the motion.

Visualizing Joint Types

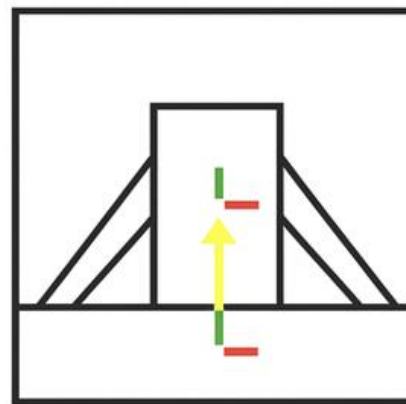
-  **Revolute:** rotates like a hinge
-  **Continuous:** spins freely (e.g., wheel)
-  **Prismatic:** slides in/out (e.g., piston)
-  **Fixed:** no motion (sensor frame or base)



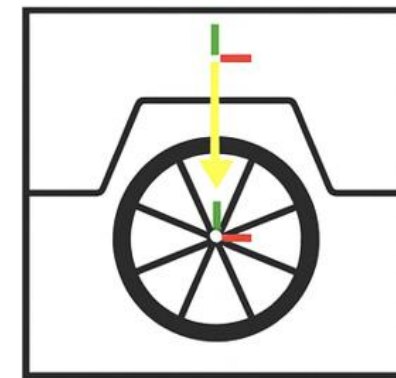
Revolute



Prismatic



Fixed



Continuous

The URDF Syntax

- URDF = Unified Robot Description Format
- Built on XML (Extensible Markup Language)
- All elements are expressed as **tags**
 - Tags can contain **attributes** and **nested tags**
- Used to describe every component of the robot
(links, joints, visuals, collisions, sensors, etc.)

XML Tag Structure

- ◆ Opening tag: `<tagname>`
- ◆ Closing tag: `</tagname>`
- ◆ Attributes: inside the tag (e.g., `<link name="base_link">`)
- ◆ Nested elements: tags inside other tags

```
<link name="base_link">  
  <visual>  
    ...  
  </visual>  
</link>
```

The Robot Tag

Every URDF file starts with:

xml

```
<?xml version="1.0"?>
<robot name="my_robot">
  ...
</robot>
```

- The **XML declaration** defines the document type
- The `<robot>` tag is the **root element** of the file
- All other tags (links, joints, etc.) go **inside the** `<robot>` tag

💡 Think of `<robot>` as the "container" for your robot definition.

Robot Name Attribute

- The `name` attribute defines the robot's identifier
- Must be **unique** within your ROS workspace
- Appears in many ROS tools like `rviz` and `gazebo`

 Example:

```
xml
```

```
<robot name="goFa_robot">
```

Syntax Summary

- ✓ URDF = XML format
- ✓ Always starts with `<?xml version="1.0"?>`
- ✓ Must have **one** `<robot>` **root tag**
- ✓ All other tags (links, joints, etc.) go inside
- ✓ Use clear **naming conventions**

The <link> Tag

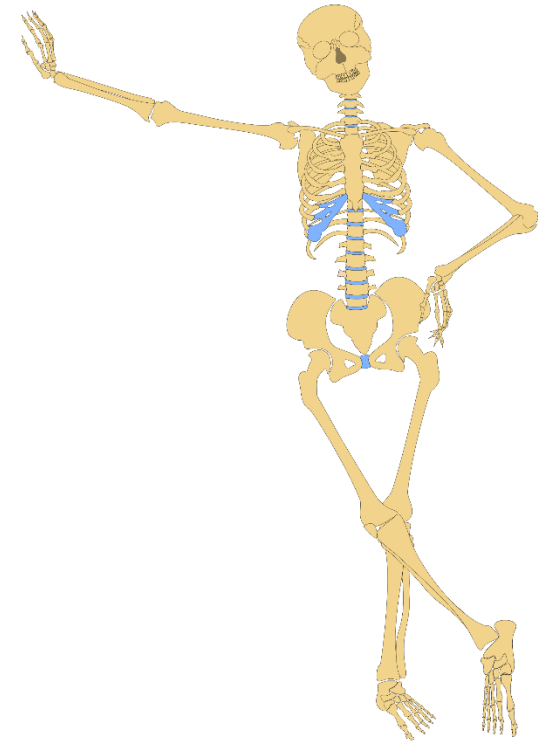
- Each **link** represents one **physical component** of the robot
- Basic structure:

```
xml

<link name="base_link">
  ...
</link>
```

- Can include **visual**, **collision**, and **inertial** properties
- Not all are required—but needed for **simulation and visualization**

✚ *Think of a link as one solid body or “bone” in your robot.*



Visual Element

Defines how the link **looks** in RViz or Gazebo.

xml

```
<visual>
  <geometry>...</geometry>
  <origin xyz="..." rpy="..." />
  <material>...</material>
</visual>
```

💡 Note: Colours affect RViz only, not Gazebo (different rendering).

Property	Purpose
Geometry	Shape (box, cylinder, sphere, or mesh)
Origin	Offset from link origin
Material	Colour or predefined material

Collision Element

Used for physics collision detection.

xml

```
<collision>  
  <geometry>...</geometry>  
  <origin xyz="..." rpy="..." />  
</collision>
```

- Often copied from the visual tag
- But may use **simpler geometry** for faster computation (e.g., box instead of a complex mesh)

⚙️ *Simpler collision = faster simulation.*

Inertial Element

Defines how the link reacts to **forces and torques**.

xml

```
<inertial>
  <mass value="1.0"/>
  <origin xyz="0 0 0" rpy="0 0 0"/>
  <inertia ixx="..." ixy="..." ixz="..." iyy="..." iyz="..." izz="..." />
</inertial>
```

Property	Description
Mass	Total mass of the link
Origin	Centre of mass
Inertia	3×3 inertia matrix (rotational resistance)

Summary: Link Tag

- ✓ Defines each rigid body of the robot
- ✓ Can contain:
 - **Visual** (appearance)
 - **Collision** (physical boundaries)
 - **Inertial** (mass + dynamics)
- ✓ Optional elements depending on application
- ✓ Supports multiple visuals or collisions

```
<link name="arm_link">  
  <visual>  
    <geometry>  
    <origin>  
    <material>  
  </visual>  
  
  <collision>  
    <geometry>  
    <origin>  
  </collision>  
  
  <inertial>  
    <mass>  
    <origin>  
    <inertia>  
  </inertial>  
</link>
```

The <joint> Tag

- Defines how two links are connected and move relative to each other
 - Each joint = one transform between a parent link and child link
 - Similar to TF transforms:
 - Links = frames
 - Joints = transforms between frames
- 💡 *Links define structure — joints define motion.*

Core Elements of a Joint

Every joint must include:

xml

```
<joint name="joint1" type="revolute">  
  <parent link="base_link"/>  
  <child link="arm_link"/>  
  <origin xyz="0 0 1" rpy="0 0 0"/>  
</joint>
```

Element	Description
name	Unique joint name
type	Type of motion (fixed, revolute, prismatic, continuous)
parent	The stationary (upstream) link
child	The moving (downstream) link
origin	Initial position/orientation offset

Additional Parameters

For movable joints, define:

xml

```
<axis xyz="0 0 1"/>  
<limit lower="-1.57" upper="1.57" velocity="1.0" effort="2.0"/>
```

Tag	Function	Units
axis	Defines motion axis	unit vector (x, y, z)
limit	Sets movement boundaries	m / rad / m·s ⁻¹ / N·m

⚙️ *Axis = direction of motion; Limit = how far and how fast.*

Common Joint Types Recap

Type	Motion	Example	Notes
Fixed	No motion	Mounting a sensor	Simplest joint
Revolute	Rotation with limits	Arm joint	Most common
Continuous	Unlimited rotation	Wheel	No angle limit
Prismatic	Linear motion	Slider	Translational movement

Example Joint Snippet

xml

```
<joint name="shoulder_joint" type="revolute">  
  <parent link="base_link"/>  
  <child link="arm_link"/>  
  <origin xyz="0 0 0.2" rpy="0 0 0"/>  
  <axis xyz="0 0 1"/>  
  <limit lower="-1.57" upper="1.57" velocity="1.0" effort="5.0"/>  
</joint>
```

 This defines a shoulder joint rotating about the Z-axis, $\pm 90^\circ$.

Summary: Joints

- ✓ Connect two links
 - ✓ Define movement type and direction
 - ✓ Include axis, limits, and origins
 - ✓ Crucial for simulation, kinematics, and TF publishing
- 💡 *Getting the joint definitions right = correct robot motion.*

Extra Tags in URDF

The core tags (`robot`, `link`, `joint`) make up most of a URDF file — but URDF also supports **additional tags** to extend functionality.

💡 Extra tags are optional and ignored by parsers that don't use them.

Common extra tags:

- `<material>` → Define and reuse colors across multiple links
- `<gazebo>` → Add simulation-specific parameters for Gazebo
- `<transmission>` → Describe actuator interfaces (drives and motors)

Material Tag Example

xml

```
<material name="grey">
  <color rgba="0.6 0.6 0.6 1.0"/>
</material>
```

- Defines a named material ("grey") that can be reused later:

xml

```
<visual>
  <material name="grey"/>
</visual>
```

 Encourages consistency and simplifies color management.

Gazebo Tag

xml

```
<gazebo>
  <material>Gazebo/Blue</material>
  <mu1>0.9</mu1>
  <mu2>0.8</mu2>
</gazebo>
```

Used to set Gazebo-specific properties like:

- Friction coefficients (`mu1` , `mu2`)
- Textures and materials
- Plugins for controllers or sensors

🌸 *Has no effect in RViz – for simulation only.*

Transmission Tag

xml

```
<transmission name="shoulder_trans">  
  <type>transmission_interface/SimpleTransmission</type>  
  <joint name="shoulder_joint"/>  
  <actuator name="motor1"/>  
</transmission>
```

Specifies how a joint is driven by an actuator.

Used by `ros_control` and Gazebo ROS controllers.

⚙ Essential for robots with motors or servo interfaces.

Naming Conventions

Consistent naming makes your URDF easier to read and debug.

✓ Best Practices:

- Pair links and joints: `arm_link`, `arm_joint`
- Keep names **unique** and **lowercase**
- Follow **ROS standards** when available:
 - Humanoid: `torso_link`, `left_arm_joint`
 - Mobile platform: `base_link`, `wheel_left_joint`

🧩 *Clear naming = clear structure = easier debugging.*

Introducing xacro

- **xacro** = XML Macro tool for ROS
 - Makes URDFs easier to read, reuse, and maintain
 - Allows:
 - ✓ Splitting large URDFs into multiple files
 - ✓ Avoiding duplicated code using macros & properties
- 💡 *Think of xacro as “programming for URDF.”*

Enabling xacro

To use xacro, add this line inside the robot tag:

```
xml
```

```
<robot xmlns:xacro="http://www.ros.org/wiki/xacro">
```

✚ This enables xacro commands and macros.

Even if you don't need xacro yet, it's good practice to include this line in all robot files.

Running xacro

- xacro processes your files into a single URDF before use
- This happens automatically when launching with `robot_state_publisher`

Example (from a launch file):

```
bash
```

```
ros2 launch my_robot display.launch.py
```

or directly:

```
bash
```

```
ros2 run xacro xacro my_robot.urdf.xacro -o my_robot.urdf
```

💡 *xacro expands everything and publishes it as one robot_description message.*

Splitting Your URDF

Large URDFs become hard to maintain → split them.

Typical file structure:

- `core.urdf.xacro` – main robot links and joints
- `materials.xacro` – colour definitions
- `sensors.xacro` – camera/LiDAR links
- `macros.xacro` – reusable templates

Include them in the main file:

xml

```
<xacro:include filename="materials.xacro"/>  
<xacro:include filename="sensors.xacro"/>
```

 *Helps version control and modular design.*

Including Files

Each included file also uses the same header:

```
xml

<robot xmlns:xacro="http://www.ros.org/wiki/xacro">
    ...
</robot>
```

If it has a name → standalone URDF

If not → treated as part of the main robot

 You can include multiple files (nested or in series).

DRY Principle (Don't Repeat Yourself)

Why avoid duplicate code?

- Reduces human error
- Easier to update shared parameters
- Prevents inconsistencies (e.g., mismatched units or speeds)

💡 *xacro automates repetition using variables and macros.*

Using Properties

Define a constant once, reuse it anywhere:

xml

```
<xacro:property name="arm_radius" value="0.5"/>
```

...

```
<cylinder radius="${arm_radius}" length="7"/>
```

 *One change updates all related components automatically.*

Mathematical Operations

xacro supports math expressions:

```
xml
```

```
<cylinder length="${4*arm_radius + pi}"/>
```

- Supports `+`, `-`, `*`, `/`, and constants like `pi`
- Combine with properties for parametric designs

 *Ideal for robots with proportional geometry.*

Macros (Reusable Templates)

Define a macro once, then reuse:

```
xml
<xacro:macro name="inertial_box" params="mass x y z *origin">
  <inertial>
    <xacro:insert_block name="origin"/>
    <mass value="${mass}"/>
    <inertia ixx="${(1/12)*mass*(y*y+z*z)}" ... />
  </inertial>
</xacro:macro>
```

Then call it like:

```
xml
<xacro:inertial_box mass="12" x="2" y="3" z="4">
  <origin xyz="0 2 4" rpy="0 0 0"/>
</xacro:inertial_box>
```

 Expands automatically into the full inertial block.

Result of Macro Expansion

xml

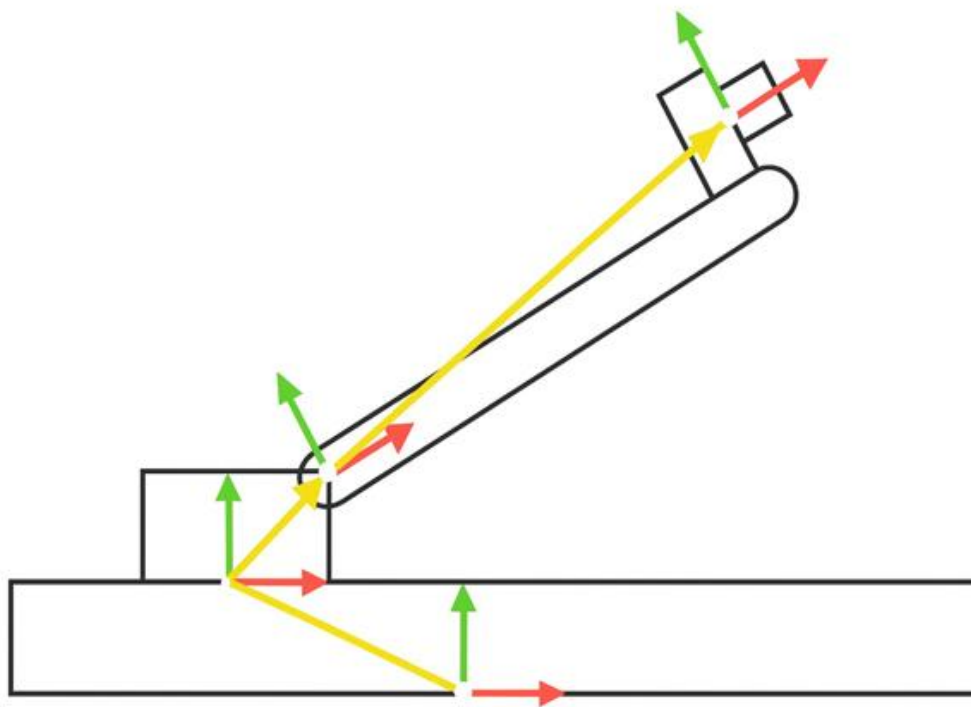
```
<inertial>  
  <origin xyz="0 2 4" rpy="0 0 0"/>  
  <mass value="12"/>  
  <inertia ixx="25" ixy="0.0" ixz="0.0"  
    iyy="20" iyz="0.0"  
    izz="13"/>  
</inertial>
```

- ✓ Saves time
- ✓ Prevents mistakes
- ✓ Simplifies complex robots

Summary: Why Use xacro?

- ✓ Makes URDFs modular
 - ✓ Avoids code duplication
 - ✓ Supports math and constants
 - ✓ Enables reusable macros
 - ✓ Required for large, multi-part robots
- 💡 *URDF + xacro = scalable robot modeling.*

Example: SliderCam Robot



```
✓ LEC8_WS [WSL: UBUNTU-24.04]
  > build
  > install
  > log
  ✓ src / urdf_example M
    > description
      ≡ example_gazebo.xacro
      ≡ example_include.xacro
      ≡ example_robot.urdf.xacro
    > launch
      📄 rsp_sim.launch.py
      📄 rsp.launch.py
  > worlds
  📄 CMakeLists.txt
  📄 package.xml
  ⓘ README.md
  ≡ view_robot.rviz
  ≡ view_with_camera.rviz
```

Build & Source Workspace

```
source /opt/ros/jazzy/setup.bash
```

```
sudo apt update && sudo apt upgrade -y
```

```
sudo apt install -y git
```

```
git --version # confirm Git is installed
```

```
cd ~
```

```
mkdir -p lec8_ws/src
```

```
cd lec8_ws/src
```

```
git clone https://github.com/malubaidy/urdf_example.git
```

```
ls
```

Build & Source Workspace

```
cd ~/lec8_ws
```

```
colcon build
```

```
source install/setup.bash
```

```
ros2 launch urdf_example rsp.launch.py
```

In a new terminal:

```
source /opt/ros/jazzy/setup.bash
```

```
Rviz2
```

RVIZ

- **Add TF in RViz2**
 - In RViz2 → Add → Select TF
 - Set Fixed Frame = world
- **Add Robot Model in RViz2**
 - In RViz2 → Add → Select RobotModel
 - Set Description Topic = /robot_description

Summary



- **URDF (Unified Robot Description Format)** → XML-based file describing a robot's structure.
- **Links** → Physical components (bodies).
- **Joints** → Connections defining movement between links.
- **Tags** → Define visual, collision, and inertial properties.
- **Extra tags** → Extend function (`material` , `gazebo` , `transmission`).
- **Naming conventions** → Keep files organized and readable.
- **xacro** → Makes URDF modular and efficient:
 - Split into multiple files
 - Reuse code with properties and macros

THANK YOU.



WE ARE

HUMBER